Aditya Muthukattu (z5422156), William Chan (z5481729), Jovan Xin (z5491897), Riley Haydon (z5416346)

NTT CORE ACCELERATION FOR POC

Accelerating Number Theoretic Transforms (NTT) for Post-Quantum Cryptography on the Kria KV260

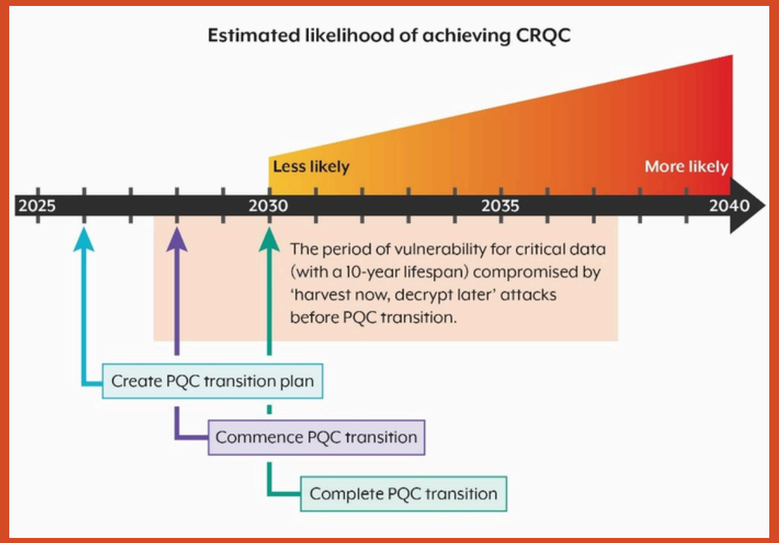
QUICK RECAP

Native Polynomial Multiplication is Inherently Slow

- Traditional asymmetric encryption uses prime factorisation and logarithms for public/private keys e.g RSA, ECC.
- Quantum computers (not yet at a threatening scale) can crack such encryption **fast** using Shor's Algorithm

Shift Towards Quantum-Resistant Cryptographic Algorithms

Regulatory body's that align with the United States National Institute of Standards and Technology (NIST) finalised Module-Lattice-Based Key-Encapsulation Mechanism (**ML-KEM**) (renamed from CRYSTALS-**Kyber**) as the standard to be implemeneted by 2030



https://www.cyber.gov.au/business-government/secure-design/quantum/planning-for-post-quantum-cryptography

25% - 90% (Really depends on the specific hardware/ software implementation targeted)

NTT Execution Time Within Kyber

Bisheh Niasar, Mojtaba & Azarderakhsh, Reza & Mozaffari Kermani, Mehran. (2021). High-Speed NTT-based Polynomial Multiplication Accelerator for CRYSTALS-Kyber Post-Quantum Cryptography.

SOLUTION APPROACH

(At a very simple level)

CREATE NAIVE/ WORKING NTT IMPLEMENTATION

- Use the online-sourced simple/naive NTT implementation and embed it into an executable HLS file/ function(s)
- Benchmark and verify performance and accuracy

OPTIMISE NTT IN HLS (AS MUCH AS POSSIBLE)

- Reduce the latency and intervale as much as possible
- Identify and try to attack bottlenecks that take up significant amounts of time (particularly like the copy in/out)

CREATE POLYMUL

- Dataflow canonical form
- Inferred parralelism where possible.
- In parallel (sort of) with optimising the NTT, create a polymul implementation firstly using an unoptimised but verified NTT

INTEGRATE NTT INTO POLYMUL AND CARRY-FORWARD OPTIMISATIONS WHERE POSSIBLE/ WITH TUNING

- Approaching the end of the project (but what was ideally during the project), integrate the optimised NTT such that polymul works
- Retrospectively, optimising the polymul itself/tuning it to work correctly required more work than anticipated

PROJECT AIMS

Bit-exact HLS NTT/INTT kernel

CORE

- Using params: n=256, q=3329 (Kyber)
- Test the HLS kernel against a reference implementation on the KV260's Cortex-A53 across random and edge-case vectors.

Complete

Every coefficient matched exactly. The arithmetic is over a finite field, a correct kernel produces identical output to software (Naive $O(n^2)$ implementation).

Full polynomial multiply, measured speedup

CORE

- Forward NTT => pointwise multiply => inverse NTT
- Measure latency and throughput on-board against the same routine running on the A53

Complete

Processing 1 polynomial at an interval defined by the unroll factor, currently set to 32 cycles.

Integrate into software KEM implementation

STRETCH

Swap software poly-multiply for the accelerator in Kyber's keygen, encaps, and decaps. Measure whole-KEM speedup to see kernel gain survives once hashing and sampling are included.

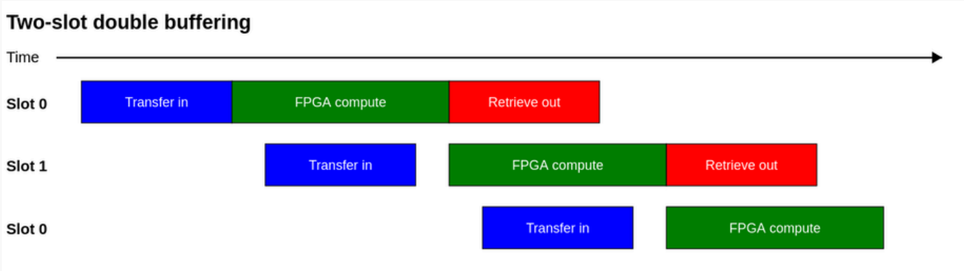
Complete

Integrating our polymul into the encryption phase of kyber, we proved the correctnees and applicability of our implementation.

HARDWARE / SOFTWARE SPLIT + RESULTS

- **FPGA:** Maximise throughput (polynomial multiplications per second)
  - AXI Burst read/write
  - Complete canonical dataflow design
  - Interval defined by unroll factor (currently new mul per 32 cycles).
    - Theoretical max throughput of 6.25M mul/s
- **Host**/Platform: Minimise cost of memory transfer (batching and double buffering)
  - Narrowed memory transfers: pack 32 * 16bit coeffs - fits in the 512bit bus
  - Separated the 3 connections (A, B, out) onto separated HP ports (avoid memory contention)
  - Batch operations (256 poly. muls per kernel launch → throuphput of **2.18M muls/s**)
  - Overlap transfer and compute (configurable ring buffer, currently set to depth of 2, sufficient to overlap transfer: **3.78M mul/s**)

Configuration	Throughput	Speedup
ARM A53 (max clock, -O3)	36,000/s	1x
FPGA, batch 1 (48us/op)	21,000/s	0.58x
FPGA, batch 64 (0.47us/op)	2.1M/s	56x
FPGA, batch 256 + ring (0.27us/op)	3.78M/s	117x



HARDWARE DESIGN

```c
// original Kyber C code
void ntt(int16_t r[256]) {
    k = 1;
    for (len = 128; len >= 2; len >>= 1) {          // ← [A] outer loop: 7 stages
        for (start = 0; start < 256; start += 2*len) {  // ← [B] blocks within a stage
            zeta = zetas[k++];
            for (j = start; j < start + len; j++) {    // ← [C] one butterfly
                t = fqmul(zeta, r[j + len]);           // multiplies two field elements and
                                                       // reduces the result mod q,
                r[j + len] = r[j] - t;                  // ← [D] writes back into r[]
                r[j]       = r[j] + t;                  // ← [D] writes back into r[]
            }
        }
    }
}
```

```c
void basemul(int16_t r[2], const int16_t a[2], const int16_t b[2], int16_t zeta) {
    r[0]  = fqmul(a[1], b[1]);      // ← [A] cross term, scaled by zeta below
    r[0]  = fqmul(r[0], zeta);      // ← [B] multiply by zeta — the constant lookup
    r[0] += fqmul(a[0], b[0]);
    r[1]  = fqmul(a[0], b[1]);
    r[1] += fqmul(a[1], b[0]);
}
```

```c
void poly_basemul(int16_t r[256], const int16_t a[256], const int16_t b[256]) {
    for (int i = 0; i < 128; i++) {             // ← [A] 128 iterations, one pair each
        int16_t z = zetas[64 + i/2];             // ← [B] independent lookup every time
        basemul(r+2*i, a+2*i, b+2*i, z);         // ← [C] one basemul call per pair
    }
}
```

```c
void poly_mul(const int16_t a[256], const int16_t b[256], int16_t r[256]) {
    ntt(a, na);            // transform a into the NTT domain
    ntt(b, nb);            // transform b into the NTT domain
    basemul(na, nb, prod);  // pointwise multiply in the NTT domain
    invntt(prod, r);        // transform the product back to normal domain
}
```

POLYMUL

BRAM: 72% **DSP: 31%**

FF: 26% **LUT: 31%**

51% of the BRAM utilisation comes from
3 AXI-burst sets of hardware

79.6X
Kernel Speedup

117.22X
End-to-End (Ring-buffer)

~3.78 MIL NTTS/S

59.53X
End-to-End (Serial)

~2.14 MIL NTTS/S

NTT CORE

BRAM: 69% **DSP: 53%**

FF: 19% **LUT: 47%**

13.3X Kernel Speedup

10.98X **~1.8 MIL NTTS/S**
End-to-End

KYBER

BRAM: 21% **DSP: 63%**

FF: 29% **LUT: 48%**

6.81X Kernel Speedup

5.93X **13,209 ENCRYPTIONS/S**
End-to-End

EVALUATION

Accelerating and Verifying the NTT

We first measure and optimised the NTT independently:

- Main computation throughout polymul
- 128 NTTs batched
- Pipelined butterfly loops (up to 32-way, but scaled down for resource utilisation for poly mul)
- 64 NTT stage memory banks (also scaled down in poly mul)

PolyMul Continued Optimisations

Pipelining down to 4-way for resource util

Balance of resource utilisation due to requirement of running concurrent NTTs.

External memory access bottleneck that we needed to address with AXI-burst

CONCLUSIONS

Accomplished

- **All of our original aims** (on board)
  - An optimised polynomial multiplication
  - Integration into encryption stage of Kyber

Future Steps

- Implementing full Kyber
  - Key generation, decryption
- Creating a batch scheduler
  - Queuing diff operations
- Add switchable 8th NTT layer for Dilithium

AI Usage

- All kinds of debugging purposes
- Assisting with understanding
- Terminal commands / scripts for running programs (eg. optimisation sweeps)
- Very handy for stubborn compiler issues

Reflections

- Pipelining was difficult (between the team!)
- Builds take forever (ram is expensive)
- Fine balance of available resources
  - Careful benchmarking required

Aditya Muthukattu (z5422156), William Chan (z5481729), Jovan Xin (z5491897), Riley Haydon (z5416346)

THANK YOU!

(when we getting a drink together Oliver?)